Multi-Agent Chance-Constrained Reinforcement Learning For Autonomous Vehicles

Awet Mekonen Araia

MSc. Thesis

July 2023

A thesis submitted to Khalifa University of Science and Technology in accordance with the requirements of the degree of Master of Science in Computer Science in the Department of Electrical Engineering and Computer Science.

Multi-Agent Chance-Constrained Reinforcement Learning For Autonomous Vehicles

by

Awet Mekonen Araia

A thesis submitted in partial fulfillment of the requirements for the degree of

Master of Science in Computer Science

at

Khalifa University

Thesis Committee

Dr. Majid Khonji (Advisor),
Khalifa University

Dr. Khaled Elbassioni (Co-Advisor),
Khalifa University

Dr. Zeyar Aung (RSC member 1),
Khalifa University

Dr. Jorge Dias- (RSC member 2),
Khalifa University

July 2023

Abstract

Awet Mekonen Araia "**Multi-Agent Chance-Constrained Reinforcement Learning For Autonomous Vehicles**", M.Sc. Thesis, M.Sc. in Computer Science, Department of Electrical Engineering and Computer Science, Khalifa University of Science and Technology, United Arab Emirates, July 2023.

Autonomous driving involves negotiating with other road users in various scenarios such as merging, overtaking, turning, and navigating unstructured urban roads. However, manually addressing all possible cases can result in a simplistic policy, and traditional methods such as optimization techniques or model-based solvers may not be feasible in real-time or may necessitate a comprehensive model to create a policy. Model-free deep reinforcement learning (DRL) offers a potential solution to these challenges. This study presents a multi-agent deep reinforcement learning approach with chance constraints for multiple autonomous vehicles in a simulated environment. The approach is formulated as a chance-constrained multiAgent Markov decision process (CC-MMDP) that limits the probability of being in a risky state by a specified threshold. The proposed approach holds promise for enhancing safety in autonomous driving while ensuring smooth traffic flow. Future work involves evaluating the approach in more complex scenarios with intersections and more autonomous vehicles.

Indexing Terms: Reinforcement learning, Multi-Agent Autonomous vehicles, collision avoidance ...

Acknowledgments

I would like to thank my family and friends for their daily support and encouragement. Their presence has been a constant source of strength. I am also thankful to my advisors, especially Dr. Majid, for their invaluable guidance and expertise throughout my project. Additionally, I would like to extend my heartfelt appreciation to all members of our research team, AV Lab, for their dedicated efforts and for creating a collaborative and friendly environment.

Declaration and Copyright

Declaration

I declare that the work in this thesis was carried out in accordance with the regulations of Khalifa University of Science and Technology. The work is entirely my own except where indicated by special reference in the text. Any views expressed in the thesis are those of the author and in no way represent those of Khalifa University of Science and Technology. No part of the thesis has been presented to any other university for any degree.

Author Name: Awet Mekonen Araia

Author Signature: 

Date: 23 / 07 / 2023

Copyright ©

Contents

List of Figures

List of Tables

List of Abbreviations

α	Learning rate
Δ	risk bound
γ	Discount factor
Π	joint policy
π	Policy
π^*	Optimal policy
θ	weights of neural networks
A	Action space
a	An action
D	Stored experience
H	Planning horizon
r	Reward
S	State space

s, s' States

T Probabilistic transition function between states

t Discrete time step

$R(s, a, s')$ Immediate reward or (expected immediate reward) the agent receives

A2C Advantage Actor-Critic

AI Arteficial Intelligence

AVs Autonomous vehicles

CC-PG Chance Constrained Policy Gradient

CTDE Centralized Training and Decentralized Execution

DDPG Deep Deterministic Policy Gradient

IA2C Indpendent Advantage Actor-Critic

IQL Independent Q-learning

MA-A2C Multi-Agent Advantage Actor-Critic

MA-TRPO Multi-Agent Trust Region Policy Gradient

MADDPG Multi-Agent Deep Deterministic Policy Gradient

MADRL Multi-Agent Deep Reinforcement Learning

MARL Multi-Agent Reinforcement Learning

MDP Markov Decision Processes

MMDP Multi Agent Markov Decision Processes

PG Policy Gradient

RL Reinforcement Learning

TRPO Trust Region Policy Gradient

CHAPTER 1

Introduction

In recent times, there has been a significant surge in the attention devoted to autonomous vehicles. The advent of self-driving cars, or autonomous vehicles, holds great potential for improving various aspects of transportation, including safety, congestion reduction, emissions reduction, and mobility enhancement. To achieve the goal of autonomous driving, a conventional framework of automation systems has been established, comprising three key components: the perception system, the decision-making system, and the execution system.

The decision-making systems encompass various components such as route planning, behavior decision, motion planning, and action selection [3]. The fundamental essence of autonomy resides in making crucial decisions, which is accomplished through the integration of planning within the navigation system, environment perception, and decision-making system [4]. The primary aim of planning is to establish a path that ensures the vehicle's safe traversal to its destination while considering aspects such as vehicle dynamics, maneuverability in the presence of obstacles, adherence to traffic regulations, and compliance with road boundaries [5].

Driving automation, as a broad concept, presents a challenge due to the complex

interactions that occur between agents. Pre-programmed rules alone cannot sufficiently address the diverse range of potential interaction mechanisms and road scenarios [6]. As a result, autonomous agents must possess the capacity to learn and navigate these complex interactions through exploration, enabling them to develop an optimal policy based on their experience with the environment. The adaptability and learning capacity of autonomous agents in response to changes in the driving environment emerges as pivotal factors in the advancement of autonomous driving systems. Recently, Deep Reinforcement Learning (DRL) has demonstrated promising advancements in the development of adaptive learning solutions specifically tailored for single-agent settings. However, there remains a significant gap in research pertaining to multi-agent settings. Limited attention has been given to exploring the potential applications and challenges of applying DRL techniques in scenarios involving multiple interacting agents. Therefore, further investigation and exploration are needed to extend the capabilities of DRL in the realm of multi-agent settings and unlock its full potential in addressing the complexities associated with autonomous driving and other domains involving agent interactions.

Multi-Agent Reinforcement Learning (MARL) refers to the complex task of learning optimal policies for multiple interacting agents using reinforcement learning (RL) [7]. Current research in autonomous driving primarily focuses on simulating road environments with human drivers. However, with the increasing deployment of autonomous vehicles, the need for a shared cooperative strategy among multiple vehicles will become of vital importance. As the number of autonomous vehicles on the road grows, a common approach to coordination and cooperation will be necessary to ensure efficient and safe operations among these vehicles.

Autonomous driving operates within a multi-agent environment, necessitating the host vehicle's adapt negotiation abilities when performing various maneuvers such as overtaking, yielding, merging, turning, and navigating unstructured urban roads [8]. Given the multitude of potential scenarios, relying solely on manual intervention to address each case would result in an overly simplistic policy. Additionally, it is crucial to strike a balance between accounting for unpredictable behavior from other drivers

and pedestrians while avoiding excessive defensiveness to ensure the smooth flow of regular traffic [8].

Coordination and cooperation among multi-agent autonomous vehicles are paramount for facilitating efficient and safe operations. By implementing cooperative strategies, vehicles can synchronize their actions, optimize traffic flow, reduce congestion, and minimize travel times through the collective sharing of information and coordinated planning. This coordination not only improves overall traffic efficiency but also enhances road safety by enabling vehicles to identify potential hazards, maintain appropriate distances, and execute maneuvers in a synchronized and harmonious manner.

The inevitability of interactions and cooperation among multiple autonomous vehicles necessitates the development of a shared cooperative strategy for effective decision-making. The multi-agent reinforcement learning (MARL) approach focuses on multi-agent issues and seeks to identify joint policies that support numerous vehicles in achieving both their individual and common objectives. One of the challenges in the deployment of large-scale AVs is safety guarantees. Given the safety-critical nature of autonomous driving, it is crucial to design MARL algorithms that incorporate safety as a constraint and prioritize achieving a high level of safety guarantee in such domains.

The main task of this research is to develop a risk-aware behavior planner for multi-agent autonomous vehicles, aiming to facilitate the learning of safe cooperative policies while satisfying safety constraints with a high probability. The study will extensively explore and compare various methodologies for extending single-agent reinforcement learning (RL) to the multi-agent context, including centralized, decentralized, and centralized training with decentralized execution. Additionally, an implementation of a chance-constrained risk-aware planner will be undertaken to ensure a high probability of satisfying safety constraints for both single-agent and multi-agent autonomous vehicles. This research endeavor represents a pivotal step toward the widespread deployment of autonomous vehicles.

1.1 Background

This section offers essential background information on reinforcement learning, covering both single-agent and multi-agent scenarios.

1.1.1 Single-Agent Reinforcement Learning

Reinforcement learning (RL) is a distinct branch of machine learning, alongside supervised and unsupervised learning. In RL, an agent learns to make optimal decisions within an environment through trial-and-error interactions. Typically, the environment is structured as a Markov decision process (MDP), where the agent's actions introduce stochastic effects on the state of the environment. RL is fundamentally driven by goals, aiming to construct a learning model that consistently achieves optimal long-term objectives through a process of trial-and-error experiences. Actions that yield improved outcomes are reinforced in the agent's behavior repertoire. This reinforcement is accomplished through a reward function, assigning positive or negative rewards at each time step. Immediate rewards carry significant weight for both the agent and the researcher. RL aims to maximize cumulative reward while considering a predetermined discount rate that signifies the importance of future rewards.

An MDP encompasses states, actions, a probabilistic transition function, a reward function, and an initial state. The objective is to determine a policy, which is a mapping from each state to an action, that maximizes the overall expected reward. Mathematically, Markov decision processes (MDPs) are formally defined as a tuple $\langle S, A, T, R, S_0, \gamma, H \rangle$where:

- S represents the finite set of states.
- A represents the finite set of actions.
- T: $S \times A \times S \rightarrow [0,1]$ is a probabilistic transition function between states, $T(s,a,s')$= $Pr(s' \mid a,s)$, where $s, s' \in S$ and $a \in A$;

- $R(s,a,s')$ is the immediate reward or (expected immediate reward) the agent receives after moving from state s to state s' by taking action a.
- $\gamma \in [0,1]$ is the discount factor
- H refers to the planning horizon

Figure 1.1 presents a graphical representation of the interaction between an agent and its environment within the framework of a Markov decision process (MDP). The MDP entails the agent receiving a reward and a state from the environment. With this information at hand, the agent selects an action, which is subsequently transmitted to the environment. The environment, in turn, responds by providing the state and reward for the next time step. This iterative process persists until the episode reaches a terminal state.

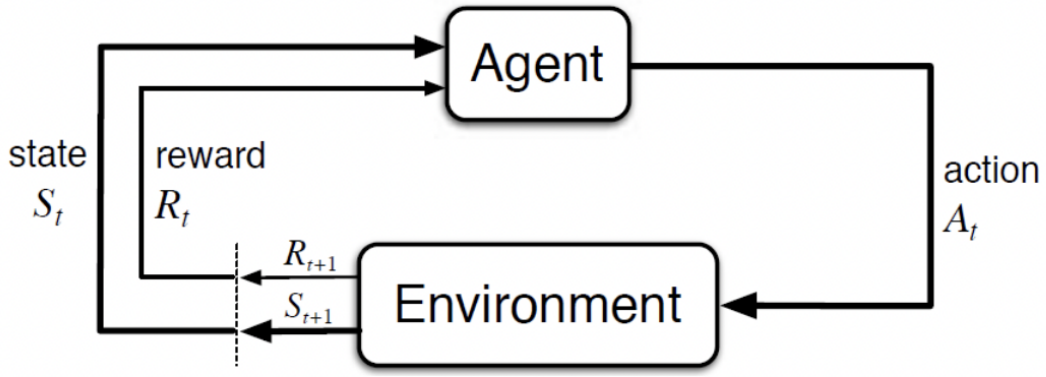

Figure 1.1: The environment in a Markov decision process

Solving Markov Decision Processes (MDPs) involves finding a policy denoted as π: $S \rightarrow A$, that determines which agent's action maximizes the discounted future reward, with a discount factor γ. A deterministic policy π maps a state into action. On the other hand, stochastic policy $\pi : S \times \{0,1,...,\text{h}-1\} \times A \rightarrow [0,1]$ maps a state to a probability distribution over actions, and $\pi(a \mid s)$ denotes the probability of selecting action a at state s. $R(s_t, a_t, s'_t)$ is the reward at time t and $V_\pi(s)$ is the expected 'return' starting at state s according to a policy π [9]. In reinforcement learning, the objective is to find an

optimal policy π^*, which results in the maximum expected sum of discounted rewards. More formally

$$V(s) = \max_{\pi} \left[\mathbb{E}\left[\sum_{t=0}^{H-1} \gamma^t R(s_t, a_t, s_{t+1}) \right] \right] \tag{1.1}$$

$$\pi^* = \arg \max_{a \in A, s \in S} \left[\mathbb{E}\left[\sum_{t=0}^{H-1} \gamma^t R(s_t, a_t, s_{t+1}) \right], \right] \tag{1.2}$$

A similar concept is the state-action Q function, $Q_\pi(s,a)$, which calculates the cumulative reward starting from state s, by selecting an action a, and from there keep following policy π.

$$Q_\pi(s,a) = \mathbb{E}_\pi \left[\sum_{t=0}^{H-1} \gamma^t R(s_t, a_t, s_{t+1}) \mid S_0 = s \right] \tag{1.3}$$

Reinforcement learning (RL) is classified into two categories: model-free and model-based techniques. In model-free reinforcement learning (RL), the objective is to learn an optimal policy directly from experience without explicitly modeling the transition probability of the environment. It focuses on estimating value or action-value functions, which predict the expected rewards associated with states or state-action pairs. On the other hand, model-based RL involves learning a model of the environment's dynamics. This learned model enables the agent to simulate trajectories and optimize its behavior accordingly, thus facilitating planning and decision-making processes.

In model-free RL, there are two notable approaches: value-based methods and policy gradient methods. Value-based methods, including Q-learning[10] and Deep Q-Networks (DQN)[11], focus on estimating value functions to learn an optimal policy. These methods aim to maximize the expected cumulative rewards by selecting actions that yield the highest values.

Q-learning [10] has gained popularity due to its successful integration with deep learning, yielding promising results. It is a value-based method that falls under the category of temporal difference (TD) learning, where the value function is approximated using bootstrapping. Q-learning is particularly suitable for non-episodic tasks as

it enables agents to learn from incomplete trajectories. However, in high-dimensional environments, classical Q-learning faces limitations as it requires maintaining a lookup table for all state-action pairs, making it impractical to cover all possible states.

$$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha \left(R_t + \gamma \max_{a \in A} \hat{Q}(s_{t+1}, a) - \hat{Q}(s_t, a_t) \right) \tag{1.4}$$

Deep Q-networks (DQN) [11] provide a parameterized solution that can effectively handle more intricate settings. In DQN, the Q-function is represented by a neural network, where the network's parameters are iteratively updated to approximate the optimal Q-values. Experience replay is employed in DQN to break correlations in the observation sequence, addressing the instability issues that can arise in parameterized Q-learning. Moreover, DQN utilizes a separate target network, with frozen weights that are periodically updated. This approach helps mitigate the moving target problem and enhances the stability of the learning process [12]. By combining these techniques, DQN demonstrates its effectiveness in tackling complex learning tasks in reinforcement learning settings. Mathematically, DQN solves the following optimization problem:

$$\min_{\theta} \mathbb{E}_{(s_t, a_t, R_t, s_{t+1}) \sim D} \left[\left(R_t + \gamma \max_{a \in A} Q_{\theta^-}(s_{t+1}, a) - Q_{\theta}(s_t, a_t) \right)^2 \right] \tag{1.5}$$

Policy gradient methods, including REINFORCE [13], Proximal Policy Optimization (PPO)[14], and Trust Region Policy Optimization (TRPO) [15] differ from value-based methods by directly optimizing the policy function using gradient ascent $\theta \leftarrow \theta + \alpha \nabla_\theta V^{\pi_\theta}(s)$. These methods iteratively update the policy parameters to maximize expected rewards and improve overall policy performance. By directly optimizing the policy, policy gradient methods can explore and improve upon a wider range of actions and potentially achieve better policies in complex environments.

$$\nabla_\theta V^{\pi_\theta}(s) = \mathbb{E}_{s \sim \mu^{\pi_\theta}(\cdot), a \sim \pi_\theta(\cdot|s)} \left[\nabla_\theta \log \pi_\theta(a|s) \cdot Q^{\pi_\theta}(s, a) \right] \tag{1.6}$$

The REINFORCE algorithm [13] estimates the action value function Q^{π_θ} in 1.6 using a sample return $R_t = \sum_{i=t}^{T} \gamma^{i-t} r_i$, which represents the cumulative sum of discounted

rewards from time step t until the end of the episode. However, this approach often suffers from high variance. To address this variance issue, one can utilize neural networks Q_ω, also known as critic networks, to estimate the true action value function Q^{π_θ} and update their parameters through TD learning. This framework is commonly referred to as actor-critic methods [16]. By incorporating the critic network, the actor-critic approach provides a more stable and effective way to estimate and optimize the action value function in reinforcement learning. Trust region methods [15] [14], PG with optimal baselines [17] [18], soft actor-critic methods [19], and deep deterministic policy gradient (DDPG)[20] methods are notable variations of actor-critic methods.

Deep reinforcement learning(DRL) is a combination of reinforcement learning and deep neural networks to address decision-making challenges within state spaces characterized by high dimensions. Deep Q-Networks (DQN)[11], Trust Region Policy Optimization (TRPO)[15], and Proximal Policy Optimization (PPO)[14] algorithms have demonstrated remarkable performance in fields such as gaming and robotic control.

1.1.2 Multi-Agent Reinforcement Learning

Until now, the emphasis has primarily been on single-agent RL, where the framework revolves around a maximum of one agent. However, the focus will now shift towards a multi-agent approach, involving at least two or more agents. Like the single-agent scenario, these agents still learn through trial and error. However, a significant distinction arises from the fact that the reward function and state spaces are affected by the collective actions of all agents. Consequently, agents are required to not only consider the environment but also consider the presence of other agents.

In multiagent reinforcement learning, autonomous agents engage in interactions with both their environment and other agents to determine the best strategies for achieving their individual objectives. While MDP is effective for modeling optimal decision-making in single-agent scenarios, they require a different approach when applied to multiagent contexts. The fundamental assumption of stationarity in Markov Decision Processes (MDPs) is compromised when the dynamics of states and the expected re-

wards are altered due to the collective actions of all agents involved. In a multi-agent environment, the manner in which agents interact with each other, whether it is co-operative, competitive, or a combination of both, along with the sequencing of their actions, has a significant impact on how the problem is conceptualized and represented. When agents have complete state observability, the problem is typically represented by a Markov game. Markov Game is an extension of the classical MDP for general multi-agent settings. Markov Game is defined as tuple $\langle N, S, \{A^i\}_{i\in N}, T, \{R^i\}_{i\in N}, S_0, \gamma, H \rangle$ where:

- $N = \{1, ..., N\}$ is the number of agents S denotes the state space observed by all agents , A^i denotes the action space of agent i .
- $A = \times_i^N A^i$ represents the finite set of joint action.
- T: $S \times A \times S \to [0,1]$ denotes the transition probability from any state $s \in S$ to any state $s' \in S$ for any joint action $a \in A$.
- $R : S \times A \times S \to \mathbb{R}$ is the reward function that determines the immediate reward received by agent i for a transition from (s,a) to s'.
- $\gamma \in [0,1]$ is the discount factor
- H refers to the planning horizon

The main difference between Markov Games and Markov Decision Processes (MDP) lies in action space A. In Markov Games, the action space is the Cartesian product of individual action spaces A^i, forming the joint-action space. In this context, the transition probabilities depend on the joint action taken by all agents.

The individual policies $\pi_i : S \times A^i \to [0,1]$ for each agent i combine to form the joint policy Π. The agents' expected return is computed based on the joint policy Π because the rewards received by the agents are influenced by the joint action taken by all agents collectively. The joint policy captures the coordinated behavior of the agents and determines their overall performance.

$$R_i^{\Pi}(x) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_t(s_t, s_{t+1}) \,|\, x_0 = x, \Pi\right] \tag{1.7}$$

Moreover, Multi-Agent Reinforcement Learning (MARL) allows for heterogeneity among agents, meaning they can have different observation and action spaces. While some MARL environments involve homogeneous agents with identical state and action spaces, such as multiple cars in an autonomous driving scenario, it is also possible for agents to have distinct reward functions. In the case of cooperative multi-agent systems, all agents share the same reward function, making the Markov Game equivalent to a Multi-Agent MDP (MMDP) [21] [22]. However, the MMDP formulation suffers from the curse of dimensionality, as the number of joint actions grows exponentially with the number of agents. In this study, the focus is on MMDPs with local interactions, where agent coordination is only required in specific interaction zones.

In multi-agent reinforcement learning (MARL), various learning paradigms exist, as illustrated by Yang and Wang [1] in Figure 1.2. These paradigms can be categorized as follows:

- Shared Policy, Independent Action: Agents share a common policy but take actions independently of each other.
- Shared Policy, Grouped Action: Agents share policies within groups, but their actions are independent of each other.
- Centralized Control: A central controller governs all agents, allowing information sharing among agents at any given time.
- Information Sharing during Training: Agents can exchange information during the training phase but not during testing.
- Local Information Sharing during Training: Agents share information with their immediate neighbors during training, but not during testing.

These paradigms represent different ways in which agents can interact, coordinate,

and share information in MARL settings, providing flexibility in designing multi-agent learning scenarios.

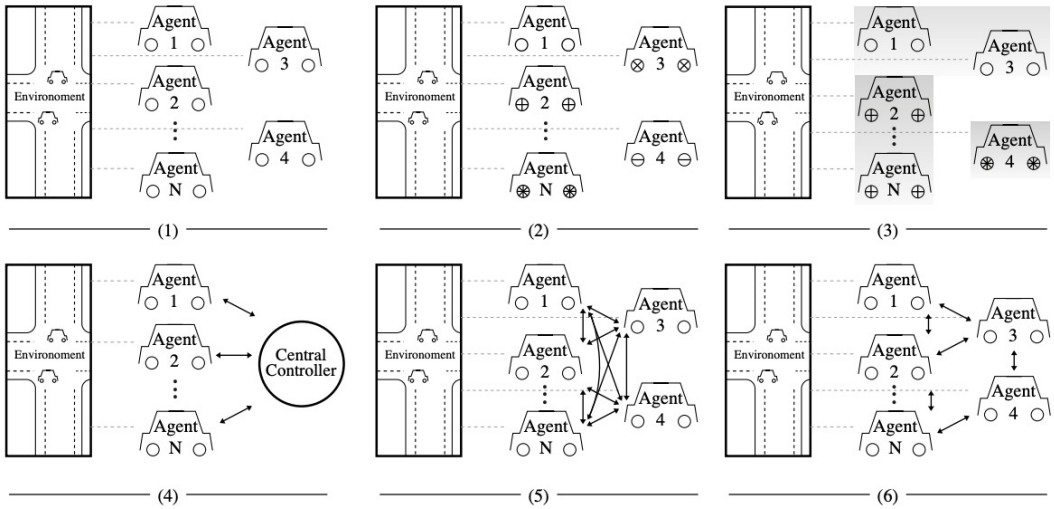

Figure 1.2: Common Learning paradigms in MARL Algorithms [1]

1.2 Motivations and Problem Description

The deployment of artificial intelligence (AI) in interconnected and cooperative applications and devices is experiencing significant growth. One area where its relevance is becoming increasingly evident is autonomous mobility and traffic scenarios. In such scenarios, effective cooperation and communication among multiple agents are crucial to finding common solutions. While single-agent approaches like reinforcement learning (RL) may not suffice in producing desired outcomes, multi-agent reinforcement learning (MARL) can address the challenges posed by multi-agent problems. MARL is particularly valuable in situations involving connected autonomous vehicles, where agents strive to achieve individual and/or common objectives within a shared environment. However, it is important to acknowledge that MARL also has its limitations, such as potential communication gaps, challenges in credit assignment to agents, and nonstationarity of the environment

Autonomous vehicles possess immense potential for enhancing overall transportation efficiency through optimized traffic flow, reduced congestion, and minimize travel

times. They have the potential to transform the way people commute, making transportation more accessible, convenient, and environmentally friendly. However, a key obstacle to the widespread deployment of autonomous vehicles (AVs) lies in ensuring safety. Technical challenges arise from the unpredictable operating environments of AVs, including road and weather conditions, perception errors, and uncertainty in pedestrian and vehicle behavior. Overcoming these challenges necessitates the development of robust AV control algorithms that account for various sources of uncertainty and generate control policies that are demonstrably safe. By addressing safety concerns, autonomous vehicles can unlock their transformative potential and revolutionize transportation as we know it.

As interactions and cooperation between connected autonomous vehicles are inevitable, a common cooperative strategy across multiple vehicles will become necessary for effective decision-making. Multi-agent reinforcement learning (MARL) approach focuses on multi-agent issues and seeks to identify joint policies that support numerous vehicles in achieving both their individual and common objectives. One of the challenges in the deployment of large-scale AVs is safety guarantees. Since Autonomous driving is a safety-critical task, designing MARL algorithms that incorporate safety as constraints are of utmost importance for such domains. Moreover, MMDP formulation of the problem suffers from a dimensionality curse as the joint actions increase exponentially with the number of agents. This paper addresses the mentioned challenges by proposing a MARL framework for multi-agent MDPs with local interactions under safety constraints. We further explore different autonomous vehicle formations that restrict the action space in a meaningful way, such as platooning.

1.3 Contributions

In light of the concerns outlined in the preceding section, we propose an enhanced approach known as chance-constrained risk-aware multi-agent deep reinforcement learning for autonomous vehicle behavioral planning. The main contributions of this thesis

are:

1. It provides an in-depth literature review of the current state of Single and Multi-Agent Reinforcement Learning methods for autonomous vehicles.

2. The thesis includes a comprehensive performance analysis of state-of-the-art model-free policy gradient reinforcement learning algorithms.

3. It involves the preparation of autonomous driving simulations specifically designed for both single-agent and multi-agent autonomous vehicles.

4. The thesis introduces a chance-constrained reinforcement learning algorithm tailored for single-agent scenarios.

5. It extends the aforementioned single-agent chance-constrained reinforcement learning algorithm to address multi-agent settings.

CHAPTER 2

Literature Review

This literature review summarizes the current deep reinforcement learning approaches for multi-agent settings. The first section overviews the Multi-agent Reinforcement Learning and the latter section describes related works that have been done in multiple autonomous vehicles.

2.0.1 Overview of MARL

Developing a successful training strategy for multiagent reinforcement learning (MARL) presents a significant challenge due to the presence of non-stationarity and partial observability. One approach to address this challenge is to employ a centralized controller that trains multiple collaborating agents, effectively reducing the problem to a single-agent setting. In this approach, agents share their observations and policies with a central controller, which then determines the actions for each agent. This strategy helps mitigate the issue of partial observability when agents have incomplete information about the environment. However, this approach can be computationally expensive, particularly in large-scale contexts. An alternative approach, known as Independent Q-learning (IQL) [23], is widely used, where each agent independently learns its own

policy while treating other agents as part of the environment. While IQL avoids the scalability issues of centralized learning, it introduces a new problem: the environment becomes nonstationary from the perspective of each agent because it contains other agents who are also learning, ruling out any convergence guarantees. Independent advantage actor-critic (IA2C) [16] and independent proximal policy optimization (IPPO) [24], like IQL, suffers from nonstationary.

Another method called centralized training and decentralized execution combines both centralized and decentralized processing [25]. During the training phase, agents have access to additional information such as observations, rewards, gradients, and parameters from other agents. However, when executing policies, agents operate based on their own local observations. This approach aims to reduce non-stationarity and partial observability by stabilizing the learning environment even when other agents' policies change.

Sunehag et al [26] proposed Value Decomposition Network (VDN) which decomposed the joint value function into individual value functions. In the execution phase, the optimal policy is determined by selecting actions greedily based on their associated Q-values. QMIX [27] enhances the performance of VDN by combining individual value functions into a joint value function using a non-linear approach, while also imposing a monotonic constraint. However, this constraint can limit the performance of collaborative agents that heavily rely on coordination. QMIX serves as the basis for various algorithms, including Weighted QMIX, which extends QMIX to non-monotonic environments by assigning higher weights to superior joint actions [28]. MAVEN (Multi-Agent Variational Exploration) addresses QMIX's inefficient exploration problem by enabling committed exploration. It allows coordinated exploratory actions to be conducted over extended time steps using a latent space for hierarchical control [29].

In [30], the author proposed Counterfactual Multiagent model (COMA) which employs a centralized critic for approximating the value of Q-function and decentralized actors for optimizing policies. Similarly, Multi-Agent Deep Deterministic Policy Gra-

dient (MADDPG)[16] extends DDPG [20] by providing the critic with supplementary information throughout training, while the actor only relies on local information. MADDPG, unlike COMA(which uses a single centralized critic for all agents), uses a separate centralized critic for each agent, allowing them to have distinct reward functions in competitive environments. While COMA is limited to discrete policies, MADDPG has the capability to learn continuous policies.

2.0.2 RL for Autonomous Vehicles

Deep reinforcement learning techniques have been widely employed in autonomous vehicle decision-making across various scenarios. One area of significant research is the double road merging problem [31] as well as highway driving scenarios [32], [33], [34], [35]. Among the commonly utilized algorithms in these studies are Deep Q-Network (DQN) [11], [36] and Double DQN [37]. Additionally, actor-critic approaches such as Asynchronous Advantage Actor-Critic (A3C) [38] and Deep Deterministic Policy Gradient (DDPG) [20] have also been extensively explored for their efficacy in addressing autonomous driving challenges. These algorithms play a crucial role in enabling autonomous vehicles to make informed decisions based on deep reinforcement learning principles. However, most of these algorithms are designed in a simplistic environment and are not suited for multi-agent settings. Moreover, those approaches do not consider safety criteria and are not evaluated for safety considerations. Several research has been done to accommodate deep reinforcement learning, such as MADDPG [16] (which extends DDPG [20]) for general multi-agent settings. However, those approaches like RL for single agent don't consider safety criteria's and are not evaluated for safety considerations. There has been interesting research on control frameworks for connected autonomous vehicles, such as platooning [24], adaptive cruise control (ACC) [39], and cooperative adaptive cruise control (CACC) [40]. However, their emphasis primarily revolved around controller design rather than decision-making. [41] Employed RL to learn traffic control strategies for a group of autonomous cars based on the expected trajectory [37]. These deep learning-based techniques, on the other hand, solely analyze

lane-keeping scenarios, with no safety guarantees. Reinforcement learning (RL) offers an exciting opportunity to acquire efficient and high-performance driving approaches for a wide range of driving scenarios. Safety, however, does not come along naturally. Constraints are an excellent approach to include safety. Safe exploration for multi-agent RL agents is crucial in areas such as this [42][43]. The constrained Markov Decision Process (CMDP) paradigm [44] is a well-known and well-studied model for reinforcement learning with constraints, in which agents must satisfy limits on auxiliary cost expectations. A specific type of constraint that arises in certain scenarios is when there is a desire to limit the probability of constraint violations below a specified threshold. This form of constraint is called ”chance constraint” [9]. [45] proposed multi-agent chance-constrained stochastic shortest path (MCC-SSP) formulation for autonomous vehicles with local interactions where agents may endure at certain interaction points.

CHAPTER 3

Methodology

3.1 Simulation Environments

The design of the environment is especially important in the context of multi-agent reinforcement learning for autonomous vehicles. In these scenarios, the behavior of the individual vehicles can have a significant impact on the overall performance of the system. A well-designed environment can provide the learning agents with realistic and informative feedback, which can help them learn to coordinate their actions and achieve their goals more effectively. Additionally, a well-designed environment can help to ensure that the learned policies are able to generalize to new situations and achieve good performance in the real world. For example, the environment could include different types of roadways, weather conditions, and traffic patterns to help the learning agents learn to navigate a wide range of situations. Overall, the design of the environment can play a crucial role in the success of reinforcement learning for multi-autonomous vehicles.

There are several top-notch open-source simulators, such as CARLA [46] and SUMO[47], as well as environments specifically made for Reinforcement Learning and multi-agent

research. However, none of these simulators are specifically designed for MARL research on autonomous vehicles. The most relevant efforts in this area are highway-env [2] and SMARTS [48]. Highway-env was chosen as a simulation environment as they could be extended to include several scenarios for Multi autonomous vehicles.

Action Space

Given the path from the starting point to the final destination, we narrow down the list of potential actions A to lane changes and selecting preferred speed. We reduce the continuous space of feasible driving speeds to a small number of discrete operations. The agent also has the option to take an idle action that keeps the current lane and target speed. The group of actions A is specified formally as

$$A = \{0 : \text{'LANE\_LEFT'}, 1 : \text{'IDLE'}, 2 : \text{'LANE\_RIGHT'}, 3 : \text{'FASTER'}, 4 : \text{'SLOWER'}\}$$

Reward

In this work, we do not prioritize solving the challenging task of selecting a realistic and optimal reward function for driving behavior. Instead, we aim to keep the reward function simple and straightforward, avoiding explicit definitions of specific driving behaviors such as maintaining a safe distance from the vehicle ahead. By doing so, we allow appropriate behavior to emerge naturally through the learning process. It is important to note that maintaining a safe distance is considered optimal not because it receives direct rewards, but because it enhances the agent's resilience against uncertain behavior from the leading vehicle, which may brake unexpectedly.

The reward function primarily focuses on two key aspects: collision avoidance and maximizing the speed of reaching the destination. Therefore, the reward function typically incorporates two fundamental components: a velocity term and a collision term.

$$R(s,a) = a\frac{(v - v_{\min})}{v_{\max} - v_{\min}} - b \times \text{collision} \tag{3.1}$$

Here, v, V_{min}, and V_{max} represent the current, minimum, and maximum speeds of the ego-vehicle. The coefficients a and b are utilized to assign weights among the aforementioned objectives within the reward function. To ensure bounded rewards, we adopt the convention of normalizing rewards within [0,1] range.

Observation

The observation space in this simulation study, commonly represented as a matrix, captures the information available to the agents before they take action. It is designed to provide the agent with the necessary information to make decisions that result in high rewards. In this study, the observation space used is called "kinematics" and is integrated into the highway-env [2]. The kinematics observation comprises a VF array, where V represents the number of nearby vehicles considered, and F denotes the number of predetermined features. In our case, F is set to 5, and the features are denoted as $Vehicle, x, y, v_x, v_y$. The Vehicle feature corresponds to the name of the car, x represents the longitudinal position, y represents the latitudinal position, v_x represents the longitudinal speed, and v_y represents the latitudinal speed. These feature values can also be normalized within a predetermined range.

3.2 Non-Constrained Deep Reinforcement Learning

Several Multi-agent deep reinforcement learning (MADRL) has been proposed for sequential decision-making in multi-agent systems. In this paper, we will focus on policy gradient Multi-Agent Reinforcement learning (PG-MADRL) approaches to train autonomous vehicles to coordinate their actions and make decisions that benefit the group as a whole. To implement this, we will use a simulation environment, such as highway-env, that represents the real-world conditions that the vehicles are likely to encounter, such as roads with other vehicles, and obstacles. The vehicles will be trained using a multi-agent RL algorithm to learn policies that enable them to effectively coordinate their actions and make decisions that lead to the best outcomes for the group. During

the training process, the vehicles will interact with the simulated environment and make decisions based on their observations, such as the positions and velocities of other objects in the environment. By comparing the performance of different multi-agent RL algorithms, we can evaluate which ones are most effective at converging on solutions that enable vehicles to navigate roads safely and efficiently. There are several ways to extend reinforcement learning (RL) algorithms for use in multi-agent settings. One approach is to train each agent independently using a decentralized single-agent RL algorithm. In this type of training system, each agent has its own neural network and learns on its own without being influenced by the other agents. Another approach is to use a centralized training and decentralized execution (CTDE) system, which combines elements of both decentralized and centralized approaches. In a CTDE system, the agents are trained using a centralized neural network, which allows them to learn from each other's experiences. However, during execution, each agent has its own neural network and makes its own decisions, which allows for individual control over their actions. This can help to improve the overall performance of the system while reducing the complexity of the learning problem. In this paper, we will explore three deep RL algorithms that can be extended for multi-agent settings using the CTDE approach: MADDPG, MA-A2C, and MA-TRPO. These algorithms are well-suited for multiagent environments and have been shown to be effective at training agents to coordinate their actions and achieve the desired outcomes.

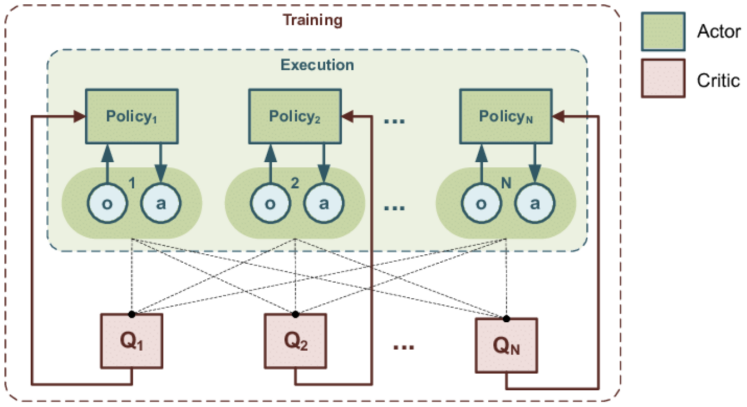

Figure 3.1: Overview of centralized training and decentralized execution approach.

3.2.1 MADDPG:

MADDPG[16] is a variant of DDPG that can be used to train a group of agents to coordinate their actions in a multi-agent environment. DDPG[20]is a model-free, off-policy reinforcement learning algorithm that can be used to train agents to learn policies that map states to actions in continuous action spaces. In a multi-agent setting, each agent learns its own policy, but must also consider the actions and behaviors of other agents. MADDPG uses deep learning and game theory to enable agents to learn policies that allow them to effectively coordinate their actions and achieve desired outcomes. MADDPG could be used to train autonomous vehicles to coordinate their actions and navigate roads safely and efficiently. In this scenario, each vehicle would be treated as an individual agent, and the MADDPG algorithm would be used to train them to learn policies that enable them to navigate roads, avoid collisions, and cooperate with other vehicles. For example, if one vehicle needs to change lanes, it could signal its intentions to the other vehicles in the vicinity using its headlights or turn signals, and the other vehicles could adjust their speeds and positions accordingly to allow the lane change to occur safely. By training the vehicles using MADDPG, it would be possible to teach them to coordinate their actions and make decisions that benefit the group, rather than just individual vehicles.

Algorithm 1 Multi-Agent Deep Deterministic Policy Gradient (MADDPG)

Initialize actor network parameters θ, critic network parameters ϕ
Set maximum number of episodes N
Initialize replay buffer D
for $i = 1$ **to** N **do**
  Initialize random process for exploration
  Receive initial state s
  Initialize episode's rewards
  for each agent j **do**
    Initialize agent's exploration noise process
  end for
  for $t = 1$ **to** T **do**
    for each agent j **do**
      Select action $a_j = \pi_{\theta_j}(s_j) + \mathcal{N}_t$
      Execute actions $\{a_1, a_2, ..., a_N\}$ and observe next state s'
      Store transition $(s, \{a_1, a_2, ..., a_N\}, r, s')$ in D
    end for
    for each agent j **do**
      if D contains sufficient samples **then**
        Sample a random minibatch of transitions $(s, \{a_1, a_2, ..., a_N\}, r, s')$ from D
        Update critic network parameters ϕ by minimizing the critic loss
        Compute actions for all agents using target actor networks: $\{a'_1, a'_2, ..., a'_N\} = \{\pi'_{\theta'_1}(s'_1), \pi'_{\theta'_2}(s'_2), ..., \pi'_{\theta'_N}(s'_N)\}$
        Update actor network parameters θ_j using the sampled policy gradient: $\nabla_{\theta_j} J \approx \frac{1}{M} \sum_m \nabla_{a_j} Q_{\phi_j}(s, \{a_1, a_2, ..., a_N\})|_{a_j = \pi_{\theta_j}(s_j)}$
        Update target networks: $\theta'_j \leftarrow \tau\theta_j + (1-\tau)\theta'_j$ and $\phi'_j \leftarrow \tau\phi_j + (1-\tau)\phi'_j$
      end if
    end for
    Accumulate rewards
    Update states: $s \leftarrow s'$
  end for
end for

3.2.2 MA-A2C:

A2C or Advantage Actor-Critic [16], is a reinforcement learning algorithm that combines the actor-critic and advantage estimation methods. It is a model-free, on-policy algorithm that can be used to train agents to learn policies that map states to actions in discrete action spaces. The actor-critic method is a type of RL algorithm that uses two separate neural networks: a "critic" network that estimates the value of a given state or action, and an "actor" network that learns a policy that maximizes the expected reward.

The critic network is trained to predict the expected return, or future discounted reward, for a given state or action, and the actor-network is trained to choose actions that maximize the predicted return. The advantage estimation method is a way of estimating the relative value, or "advantage," of different actions in each state. This allows the agent to learn policies that prioritize actions that are more likely to lead to high rewards, while still considering the long-term consequences of its actions. In the A2C algorithm, these two methods are combined to enable the agent to learn more effective policies. The actor-network is trained using the advantage estimates, which helps it to focus on actions that are more likely to lead to high rewards. The critic network is also trained using the advantage estimates, which helps it to provide more accurate predictions of the expected return. This enables the agent to learn policies that balance short-term rewards with long-term consequences, and that can adapt to changing environments over time. The comprehensive algorithm is outlined below.

Algorithm 2 Advantage Actor-Critic (A2C)

Initialize policy parameters θ and value function parameters w
Set maximum number of episodes N
for $i = 1$ **to** N **do**
    Initialize environment and state s
    Reset episode's rewards and log probability buffer
    repeat
        Select action a from policy $\pi(\theta)$
        Take action a, observe reward r and new state s'
        Store log probability of action a in buffer
        Compute TD error $\delta = r + \gamma V(s', w) - V(s, w)$
        Accumulate rewards
        Update state $s \leftarrow s'$
    until episode terminates
    Compute advantages A_t for each timestep t
    Update value function parameters: $w \leftarrow w + \alpha \sum_t \delta_t \nabla_w V(s_t, w)$
    Update policy parameters: $\theta \leftarrow \theta + \beta \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) A_t$
end for

To extend the A2C into multi-agent settings, we can use a variant of the A2C algorithm that includes a centralized critic network and decentralized actor networks. The centralized critic network is used to estimate the value of different states and actions and to provide feedback to the actor networks. The 19 actor networks are trained using

the estimates from the centralized critic network, which enables them to learn policies that consider the actions and experiences of other agents in the environment. During execution, each agent uses its own actor-network to make decisions based on its observations of the environment. The agents can communicate with each other using some form of signaling or messaging, which allows them to coordinate their actions and make decisions that benefit the group. By using CTDE with the A2C algorithm, it is possible to train a group of agents to effectively coordinate their actions and make decisions that lead to the desired outcomes in a multi-agent setting.The comprehensive algorithm is outlined below.

Algorithm 3 Multi-Agent Actor-Critic (MAAC)

Initialize centralized critic network parameters ϕ, decentralized actor network parameters θ
Set maximum number of episodes N
Initialize replay buffer D
for $i = 1$ **to** N **do**
  Initialize random process for exploration
  Receive initial states s
  Initialize episode's rewards
  for each agent j **do**
    Initialize agent's exploration noise process
  end for
  for $t = 1$ **to** T **do**
    for each agent j **do**
      Select action $a_j = \pi_{\theta_j}(s_j) + \mathcal{N}_t$
    end for
    Execute actions $\{a_1, a_2, ..., a_N\}$ and observe next states $\{s'_1, s'_2, ..., s'_N\}$ and rewards $\{r_1, r_2, ..., r_N\}$
    Store transition $(s, \{a_1, a_2, ..., a_N\}, r, \{s'_1, s'_2, ..., s'_N\})$ in D
    for each agent j **do**
      if D contains sufficient samples **then**
        Sample a random minibatch of transitions $(s, \{a_1, a_2, ..., a_N\}, r, \{s'_1, s'_2, ..., s'_N\})$ from D
        Compute centralized Q-values using the centralized critic network:
        $Q_\phi(s, \{a_1, a_2, ..., a_N\})$
        Compute decentralized target actions using target actor networks:
        $\{a'_1, a'_2, ..., a'_N\} = \{\pi'_{\theta'_1}(s'_1), \pi'_{\theta'_2}(s'_2), ..., \pi'_{\theta'_N}(s'_N)\}$
        Compute centralized target Q-values using the centralized target critic network:
        $Q'_{\phi'}(s', \{a'_1, a'_2, ..., a'_N\})$
        Compute centralized target Q-value targets:
        $y_i = r_i + \gamma Q'_{\phi'}(s', \{a'_1, a'_2, ..., a'_N\})$
        Update centralized critic network parameters ϕ by minimizing the critic loss:
        $L = \frac{1}{M} \sum_i (y_i - Q_\phi(s, \{a_1, a_2, ..., a_N\}))^2$
        Update decentralized actor network parameters θ_j using the sampled policy gradient:
        $\nabla_{\theta_j} J \approx \frac{1}{M} \sum_m \nabla_{a_j} Q_\phi(s, \{a_1, a_2, ..., a_N\})|_{a_j = \pi_{\theta_j}(s_j)}$
        Update target networks: $\theta'_j \leftarrow \tau\theta_j + (1-\tau)\theta'_j$ and $\phi' \leftarrow \tau\phi + (1-\tau)\phi'$
      end if
    end for
    Accumulate rewards
    Update states: $s \leftarrow \{s'_1, s'_2, ..., s'_N\}$
  end for
end for

3.2.3 MA-TRPO

Trust Region Policy Optimization (TRPO)[15] is a model-free, on-policy reinforcement learning algorithm that can be used to train agents to learn policies that map states to actions in continuous action spaces. It is an improvement over the popular Policy Gradient (PG) algorithm [16] [49], which has several limitations that can make it difficult to apply to complex problems. One of the key features of TRPO is its use of a trust region constraint, which helps to ensure that the updates to the policy are well-behaved and do not cause the performance of the policy to degrade. This is achieved by limiting the size of the updates to the policy, which helps to prevent the algorithm from making large, potentially harmful changes to the policy. Another key feature of TRPO [15] is its use of a natural policy gradient, which provides a more efficient and accurate way of updating the policy compared to the standard PG algorithm. The natural policy gradient considers the curvature of the value function, which can help the algorithm to make more effective updates to the policy. Mathematically, TRPO aims to optimize Equation 3.2

$$\max_{\theta} \mathbb{E}_{s_t,a_t\sim\pi_\theta}\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}A^{\pi_{\theta_{\text{old}}}}(s_t,a_t)\right] \tag{3.2}$$

Subject to

$$\mathbb{E}_{s_t,a_t\sim\pi_\theta}\left[\text{KL}\left(\pi_{\theta_{\text{old}}}(\cdot|s_t),\pi_\theta(\cdot|s_t)\right)\right] \leq \delta$$

The comprehensive TRPO algorithm is outlined below.

Algorithm 4 Trust Region Policy Optimization (TRPO)

Initialize policy parameters θ
Set maximum number of iterations N
for $i = 1$ **to** N **do**
    Collect trajectories using policy $\pi(\theta)$
    Compute advantages A_t for each timestep t
    Compute policy gradient $\hat{g} = \frac{1}{T}\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)A_t$
    Compute Fisher matrix F and its inverse F^{-1} using trajectories
    Compute natural policy gradient direction $\tilde{g} = F^{-1}\hat{g}$
    Compute step size η using line search or other method
    Update policy parameters: $\theta \leftarrow \theta + \eta\tilde{g}$
end for

Similarly, we can extend TRPO into multi-agent settings using centralized training and decentralized approach (CTDE). By using CTDE with the TRPO algorithm, it is possible to train a group of agents to effectively coordinate their actions and make decisions that lead to the desired outcomes in a multi-agent setting. MA-TRPO is a multi-agent extension of TRPO that uses a centralized training and decentralized execution (CTDE) approach. In this approach, the agents are trained using a centralized neural network, which enables them to learn from each other's experiences and improve their policies over time. During execution, each agent has its own neural network and makes its own decisions, which allows for individual control over their actions. This can help to improve the overall performance of the system while reducing the complexity of the learning problem.

Algorithm 5 Multi-Agent Trust Region Policy Optimization (MATRPO)

Initialize policy parameters θ
Set maximum number of iterations N
for each agent j **do**
    Initialize agent's value function parameters w_j
end for
for $i = 1$ **to** N **do**
    Collect trajectories using policy $\pi(\theta)$
    Compute advantages A_t^j for each agent j and timestep t
    Compute policy gradient $\hat{g}_j = \frac{1}{T}\sum_t \nabla_{\theta_j} \log \pi_{\theta_j}(a_t^j|s_t^j)A_t^j$ for each agent j
    Compute Fisher matrix F_j and its inverse F_j^{-1} for each agent j using trajectories
    Compute natural policy gradient direction $\tilde{g}_j = F_j^{-1}\hat{g}_j$ for each agent j
    Compute step size η using line search or other method
    Update policy parameters: $\theta_j \leftarrow \theta_j + \eta\tilde{g}_j$ for each agent j
    Update value function parameters w_j using TRPO update or other method for each agent j
end for

3.2.4 Comparison Methodology

The performance of the three models was evaluated using the following setup:

- Each model was implemented from scratch and adapted to a multi-agent setting using the CTDE approach
- The models were tested on a custom highway environment, which was expanded

to incorporate scenarios for multi-agent autonomous vehicles.

- Three distinct scenarios were created to assess the performance of each model: two scenarios involved only autonomous vehicles in the simulation environment, while the third scenario included multiple random vehicles to simulate human driving patterns.

- The tests were conducted using a Core i9 PC equipped with an RTX 2080 GPU.

We evaluated the three models on a custom simulation environment based on the open-source highway-env [2]. We constructed three driving scenarios with different levels of complexity. In each scenario, the objective for the agent is to successfully navigate along a predetermined route without encountering any collisions.. In the first scenario, we introduced 10 autonomous vehicles on the highway environment, where each vehicle has its own observation of the environment, and included an additional of 5 uncontrolled vehicles that represented human-driven vehicles. In the second scenario, we increased the number of autonomous vehicles to 20 to test the scalability of each algorithm. In the third scenario, we introduced an additional 20 human vehicles. By conducting these experiments, we could assess the capabilities of the three algorithms in successfully completing the driving scenarios and ensuring safe and coordinated actions among the vehicles.

3.2.5 Comparsion Results

The primary objective of this comparative analysis is to identify and select the most appropriate algorithms to use in comparison with our suggested model. Moreover, this analysis will give us insights into their strengths and limitations. We want to uncover algorithms with positive performance features, including convergence, coordination, and collision avoidance.

Scenario	MADDPG	MA-A2C	MA-TRPO
Scenario 1	4.00±0.4	27.7±0.004	28.9±0.3
Scenario 2	3.00±0.4	27.6±0.004	28.3±0.3
Scenario 3	3.00±0.9	8.2±0.9	14±0.3

Table 3.1: Average Returns

3.3 Chance Constrained Deep Reinforcement Learning

Reinforcement learning is not yet completely developed or equipped to be readily used as an ”off-the-shelf” solution for safety-critical real-world tasks. The commonly used optimization criterion for MDPs is risk neutral. In this approach, the policy's quality is evaluated by computing the total accumulated reward and maximizing the expected value of this quantity through updates. However, this approach may not be suitable for learning risky tasks, such as controlling a self-driving car in an urban setting. In such cases, it is necessary to use sophisticated control systems that ensure safety by considering the inherent uncertainty and variability in the environment. The constrained Markov Decision Process (CMDP) paradigm is a well-known and well-studied formulation for reinforcement learning with constraints [44]. CMDP restricts the expected cumulative cost to remain below a predetermined limit. Several popular safe RL algorithms, such as constrained policy optimization (CPO)[50], projection-based constrained policy optimization (PCPO)[51], and RCPO [52], are built upon this framework. However, these techniques ensure constraint satisfaction on average, which is insufficient for safety-critical applications since it allows for violations up to 50% of the time.

Another type of safety constraint used to limit the probability of constraint violations to a specified threshold is a chance constraint. Chance constraints guarantee that a specific constraint is satisfied with a predetermined probability, providing the advantage of limiting the probability of unsafe events occurring. This enables a higher probability of satisfying constraints, rather than relying solely on expectation. They can also be customized to fit different tasks and user requirements, making them well-suited for various real-world applications. Chance constraints are an essential tool for ensuring

safety in self-driving automobiles, as they enable the system to consider the uncertainty and unpredictability inherent in the environment. In the context of self-driving cars, a chance constraint is a probabilistic restriction that defines the maximum allowable probability of violating a safety constraint, such as exceeding a prescribed speed limit or colliding with an obstacle. By employing chance constraints, the self-driving car can make safe decisions with high probability and avoid situations that may lead to accidents or other hazards.

In this work, we consider chance constraint reinforcement learning for both single-agent autonomous vehicles and multi-agent autonomous vehicles. we formulate the safety constraints by imposing a lower bound on the probability of the remaining in the safe state for all times.

3.3.1 Chance Constraint For Single Agent

In single-agent chance-constrained reinforcement learning, we aim to find an optimal policy π that accomplishes the following:

1. Joint chance constraint satisfaction

   The probability that an agent avoids risky states $s_t \subseteq S_{risky}$ or stays within safe feasible states $s_t \subseteq S_{safe}$ over the planning horizon H is at least $1-\Delta$, where $\Delta \in [0,1]$ is predefined risk bound.

2. Maximization of the expected cumulative reward

$$\max_{\pi} \mathbb{E}_{\sim \pi_\theta(a|s)} \left[\sum_{t=0}^{H-1} R(s_t, a_t . s_{t+1}) \right] \tag{3.3}$$

chance constraint reinforcement learning can then be formulated as

$$\max_{\pi} \mathbb{E}_{\sim \pi_\theta(a|s)} \left[\sum_{t=0}^{H-1} R(s_t, a_t . s_{t+1}) \right] \tag{3.4}$$

Subject to

$$Pr\left\{ \bigwedge_{t=0}^{H-1} s_t \in S_{safe} \mid S_0 \right\} \geq 1-\Delta \tag{3.5}$$

Solving probability or chance constraints is not a straightforward task. In order to address this optimization problem, we need to reformulate the chance constraint as an expectation of the cumulative of indicator random variables. By doing this, we can then apply a Lagrangian-based approach. Let $I_t(s)$ represent a random indicator variable for failure at state s and time t, such that $I_t(s) = 1$ if and only if it is in a risky state, zero otherwise.

The above optimization could be approximated using Boole's Approximation as follows.

$$\max_{\pi} \mathbb{E}_{\sim \pi_\theta(a|s)} \left[\sum_{t=0}^{H-1} R(s_t, a_t.s_{t+1}) \right] \tag{3.6}$$

subject to

$$\mathbb{E}_{\sim \pi_\theta(a|s)} \left[\sum_{t=0}^{H-1} I_t(s_t) \mid S_0 \right] \leq \Delta \tag{3.7}$$

To address the optimization problem mentioned earlier, we initiate the process by constructing Lagrangian relaxation for the problem. Our methodology is based on the primal-dual formulation. We introduce a multiplier $\lambda \geq 0$ and define the corresponding Lagrangian as :

$$V(\theta) = \mathbb{E}_{\sim \pi_\theta(a|s)} \left[\sum_{t=0}^{H-1} \gamma^t R(s_t, a_t.s_{t+1}) \right] \tag{3.8}$$

$$C(\theta) = \mathbb{E}_{\sim \pi_\theta(a|s)} \left[\sum_{t=0}^{H-1} I_t(s_t) \mid S_0 \right] \leq \Delta \tag{3.9}$$

$$\mathscr{L}(\theta, \lambda) = V(\theta) - \lambda * C(\theta) \tag{3.10}$$

Chance Constraint Policy Gradient Algorithm

In this section, we introduce a policy gradient algorithm that can be used to solve the optimization problem. The algorithm works by descending in the variables θ, while

ascending in the variable λ, using the gradients of the Lagrangian function $\mathscr{L}(\theta,\lambda)$ with respect to θ. We can use any PG method to update the parameters of the policy neural network. Algorithm 6 extends existing policy gradient algorithms like REINFORCE, A2C, and TRPO. It incorporates chance constraints into the optimization process, allowing for policies that satisfy certain probabilistic constraints. By following the steps outlined in Algorithm 6, these existing algorithms can be extended to incorporate chance constraints. In this work, we specifically focused on the REINFORCE and TRPO algorithms. As a result, we introduced the CC-REINFORCE and CC-TRPO variants, which enable the optimization of policies while satisfying the desired probabilistic constraints.

Algorithm 6 Chance Constraint policy gradient (CC-PG)

Input : $\pi(|;\theta_0),\Delta,\lambda_0,H,\alpha,\beta$

for $k = 0,1,....$ **do**

  Simulate N trajectory τ with current policy $\pi_{\theta k}(a|s)$

  $prob = \frac{n^0 riskystate}{N}$

  θ **update :***use any PG algorithim*

  λ **update :** $\lambda_{k+1} = \lambda_k + \beta(prob - \Delta)$

end

3.3.2 Experiment

In this section, we aim to showcase the effectiveness of our chance-constrained policy gradient algorithm through its performance evaluation on the Highway-Env simulation environment. We chose this environment primarily because of its compatibility with widely used deep reinforcement learning (DRL) libraries. Highway-Env comprises various simulation environments that represent different driving tasks and scenarios. For this experiment, we selected two driving tasks, namely highway and two-way.

Highway

In this scenario, the ego-vehicle operates on a multilane highway with other vehicles present. The objective of the agent is to achieve a high speed while ensuring the avoidance of collisions with neighboring vehicles. Additionally, the agent receives a reward

for driving on the right side of the road. The environment provides an observation of the vehicle. This information is used as the state of the vehicle. The information received is the position, speed, and presence of all vehicles near the ego vehicle.

The available actions for the ego-vehicle in the Highway-Env simulation environment are discrete and consist of five options. These include switching to the left or right lane, slowing down, speeding up, or maintaining the current speed.

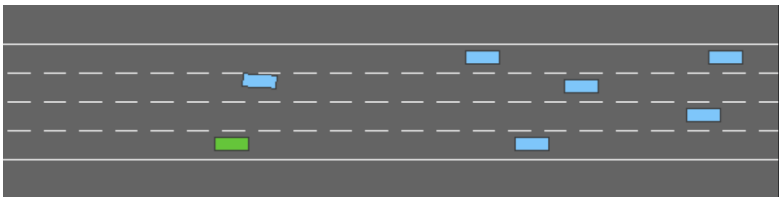

Figure 3.2: Highway-Env : Highway [2]

A Cost function was introduced to evaluate to 1 in case of a collision and 0 otherwise. This cost function aids in identifying risky states where collisions occur, and the agent incurs a cost of 1. The objective is to incorporate safety into the chance constraint by formulating it in a way that the probability of a collision for a given trajectory remains below a pre-defined threshold. Additionally, in this experiment, it was considered that the simulation terminates once the agent collides.

In this experiment, we assessed the performance of our algorithm using three different thresholds: $\Delta = 0.1$, $\Delta = 0.01$, and $\Delta = 0.05$. This implies that our algorithm was expected to meet the corresponding chance constraints of 90%,99%, and 95%.

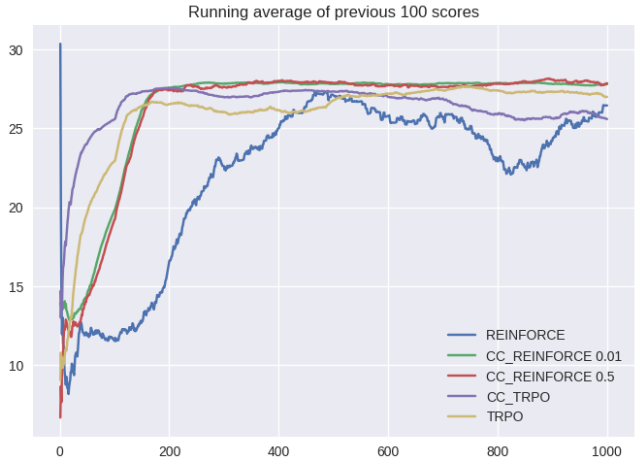

Figure 3.3: Average Cumulative Return

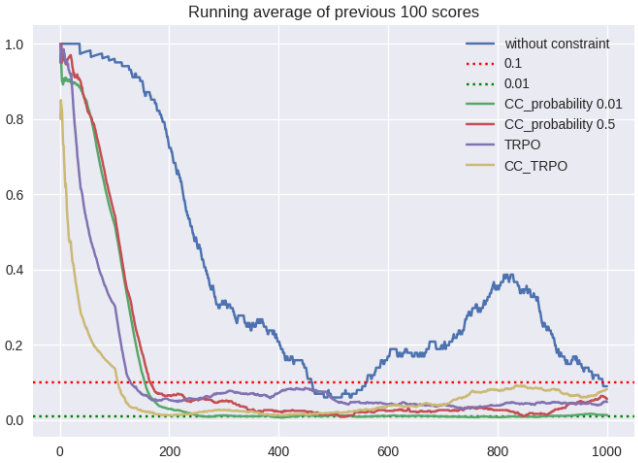

Figure 3.4: Safety probability.

Figure 3.3 show the average cumulative rewards for three different thresholds: $\Delta = 0.1$, $\Delta = 0.01$, and $\Delta = 0.05$. Our evaluations demonstrate in all thresholds, our algorithm was able to complete the driving tasks and coordinate the actions of the vehicles to avoid collisions and accumulate high cumulative rewards. After training the three models, we evaluated their performance in several different episodes. Almost always, CC-REINFORCE satisfied the safety constraints given by the predefined thresholds as shown in Table 3.2 and Table 3.3. However, there were some instances where the algorithm failed to satisfy the threshold by small amounts, specifically in 500 iterations for a threshold of 0.01. To address this issue, we can use probability-tightening techniques.

Runs	Num of constraint violation	# collision in evaluation	Probability of constraint violation
10	0	0	0.0
20	0	0	0.0
50	0	0	0.0
100	4	4	0.04
200	4	4	0.02
500	7	7	0.014

Table 3.2: Evaluation Summary for CC-REINFORCE $\Delta = 0.1$

Runs	Num of constraint violation	# collision in evaluation	Probability of constraint violation
10	0	0	0.0
20	0	0	0.0
50	0	0	0.0
100	1	1	0.01
200	1	1	0.005
500	6,4	6,4	0.012,0.008

Table 3.3: Evaluation Summary for CC-REINFORCE $\Delta = 0.01$

Two-way:

In this task, the agent is driving on a two-way road with a car in front of it. The agent can choose to either stay behind the car, which is a safe and slow option or overtake the car, which is a risky and faster option. the type of observation used in this task is an array that provides an estimate of the time-to-collision for other vehicles on the same road as the ego-vehicle. These estimates are generated for various values of the ego-vehicle speed and the lanes surrounding the current lane. They are then represented as one-hot encodings over discretized time values (bins) with 1-second increments.

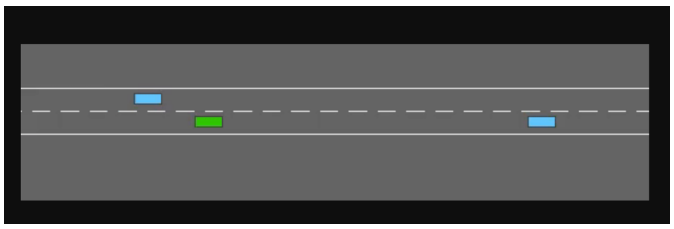

Figure 3.5: Highway-Env : Two-way [2]

In this task, we aim to evaluate our algorithm's ability to balance task accomplishment and safety while navigating the road. The agent is rewarded for overtaking the

vehicles in front of it, which is a risky option. Our goal is to train the agent to overtake the vehicle in front of it while avoiding collisions with vehicles moving in the opposite direction.

For this experiment, we utilized two policy gradient algorithms, Reinforce and TRPO, and augmented them with a chance constraint using our algorithms. To analyze the experiment, we also included Reinforce and TRPO without the chance constraint. All the algorithms are presented in the figures below. While both TRPO and Reinforce achieve high cumulative rewards, their collision probability is also high. In contrast, when using the chance-constrained CC-Reinforce and CC-TRPO, the probability of collision is below the given threshold, and they achieve a reasonable average cumulative reward.

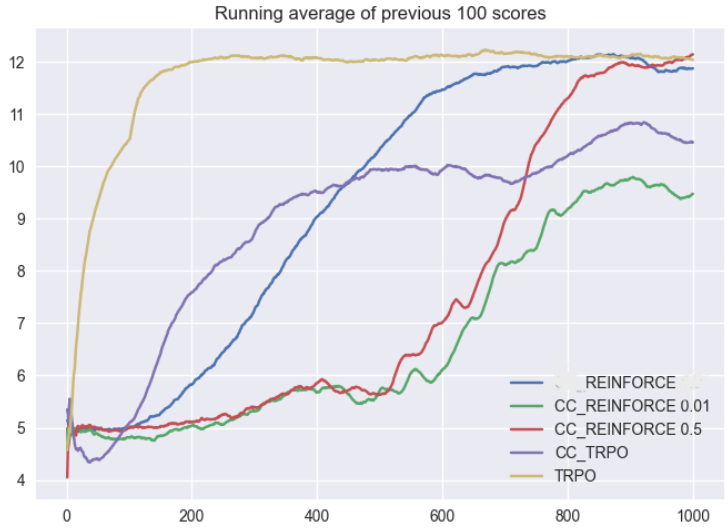

Figure 3.6: Average Cumulative Return

Table 3.4 illustrates the result of our evaluation of the algorithms in terms of average returns, number of collisions, and probability of collision, in over 10 seeds 500 iterations with a 90% confidence interval. Both REINFORCE and TRPO algorithms achieved an average of 12.0, while CC-REINFORCE with $\Delta = 0.1$, as well as CC-TRPO with $\Delta = 0.1$, had slightly lower averages of 8.9. However, when it comes to the number of collisions, all algorithms without chance constraint, as well as CC-REINFORCE with delta 0.5, had an average of 200 collisions. In the case of stochastic Primal-Dual,

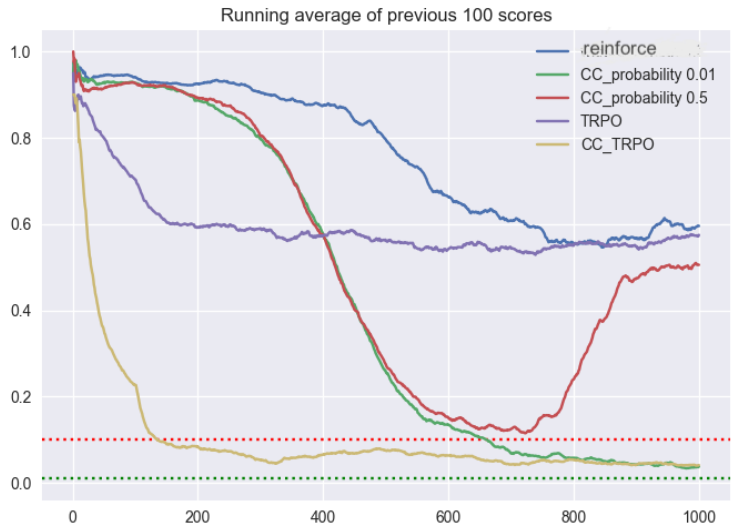

Figure 3.7: Probability of collision

even though the number of collisions is low, it achieves lower cumulative returns, indicating a conservative behavior.

Algorithms	Cumulative Returns	# Collisions	Probability of Collision
REINFORCE	12.2±0.002	285±6	0.578±0.002
TRPO	12.13±0.04	269±5	0.517±0.004
CPO	8.709±0.0003	53±3	0.1042±0.01
Stochastic Primal-Dual	0.41±0.0003	48±3	0.10±0.01
CC-REINFORCE $\Delta = 0.5$	12.21±0.03	253±3	0.49±0.003
CC-REINFORCE $\Delta = 0.1$	11.04±0.02	36±2	0.082±0.009
CC-TRPO	10.56±0.03	20±3	0.04±0.004

Table 3.4: Two-way Evaluation Result

3.3.3 Chance constraint For Multi Agent

The centralized training and decentralized execution (CTDE) approach can be an effective way to extend our chance constraint policy gradient algorithm to multi-agent settings. One way to implement this is through Multi-Agent Advantage Actor-Critic (MA-A2C) or Multi-Agent Trust Region Actor-Critic (MA-TRPO) where the chance constraint is augmented to account for the joint probability of collision across all agents. In this approach, during training, a centralized critic is used to estimate the joint value function of all agents, while each agent maintains its own actor policy. During exe-

cution, each agent only observes its local state and applies its own policy. By using this approach, agents can learn to cooperate with each other to achieve the task while satisfying safety constraints.

Highway

In our evaluation of CC-MA-A2C, we utilized a custom simulation environment based on the open-source Highway-Env [2]. We designed three driving scenarios of varying complexities to assess the performance of the algorithm in enabling each agent to complete a predetermined route without colliding with other vehicles.

The first scenario involved 10 autonomous vehicles, with each vehicle having its unique observation of the environment. The second scenario had 10 autonomous vehicles, including 5 uncontrolled vehicles that represented human-driven vehicles, to test the scalability of each algorithm. Each vehicle had its own observation of the environment.

In the third scenario, we further increased the complexity of the environment by adding 20 additional autonomous vehicles and 10 human-driven vehicles, which resulted in a total of 50 vehicles on the road. This test aimed to determine whether the algorithm could coordinate the actions of multiple agents to avoid collisions and successfully complete the driving scenarios.

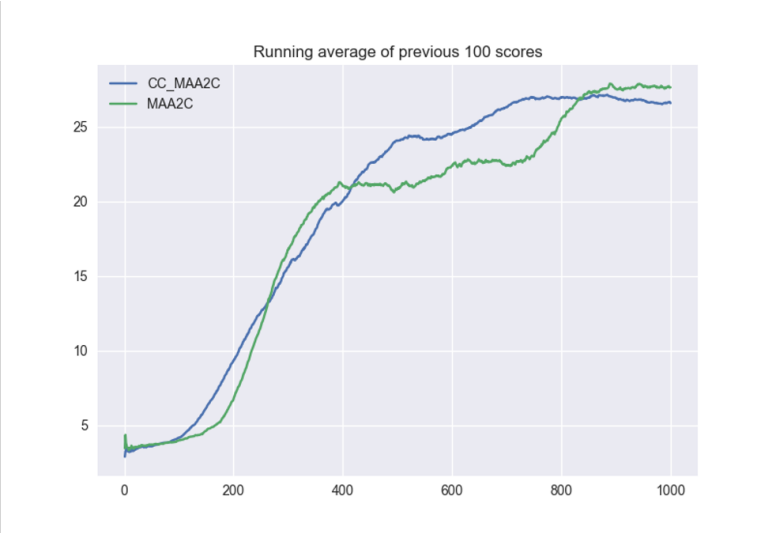

Figure 3.8: Scenario 1: Average Cumulative Return

Figure 3.9: Scenario 1: Probability of collision

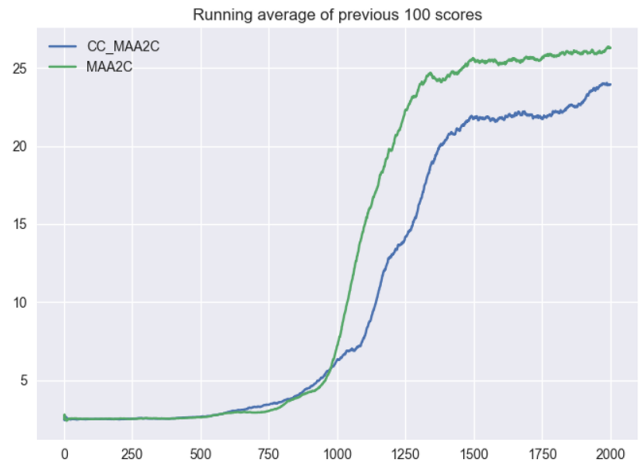

Figure 3.10: Scenario 2: Average Cumulative Return

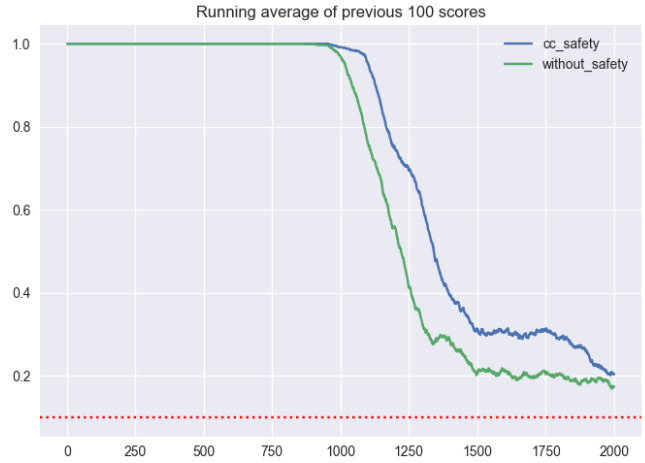

Figure 3.11: Scenario 2: Probability of collision

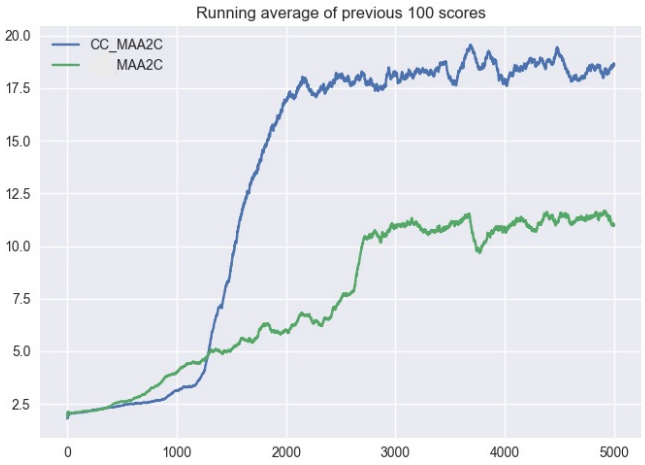

Figure 3.12: Scenario 3: Average Cumulative Return

Figure 3.13: Scenario 3: Probability of collision

Figures 3.8, 3.10, and 3.12 show the average cumulative rewards for 1000 episodes of scenario 1, 2000 episodes of scenario 2, and 5000 episodes of scenario 3, respectively. The duration of each episode's timestep depends on the presence or absence of collisions. If there are no collisions, the timestep spans 40 units. However, if a collision occurs, the timestep continues until the collision is resolved. Our evaluations demonstrate that in all scenarios, CC-MAA2C was able to complete the driving tasks and coordinate the actions of the vehicles to avoid collisions and accumulate high cumulative rewards with a low probability of collision. In the next section, we will evaluate the algorithm in an intersection environment, where coordination between autonomous vehicles is required.

Intersection

To further investigate the effectiveness of our Chance constrained multi-agent reinforcement learning approach, we conducted an experiment in an intersection scenario. Intersections are critical environments where coordination plays a vital role in ensuring the safety and efficiency of vehicle movements. At an intersection, multiple autonomous vehicles (AVs) may converge, and it is essential for these vehicles to cooperate and coordinate their actions to avoid collisions and maintain smooth traffic flow. By enabling effective cooperation and coordination among the agents, potential collisions can be

avoided, ensuring both safety and efficient traffic movement. The significance of this experiment lies in its ability to assess and analyze the performance of multi-agent reinforcement learning (MARL) algorithms specifically in the context of safe coordination, cooperation, and collision avoidance within an intersection scenario. This experiment allows us to evaluate the capabilities of our Chance constrained approach in effectively addressing these challenges. Through this experiment, we aim to gain deeper insights into the performance of MARL algorithms when applied to coordination and cooperation among AVs in intersection scenarios. By assessing the effectiveness of our approach, we can contribute to the advancement of safe and efficient autonomous transportation systems.

In this experiment, our aim was to evaluate the performance of our algorithm in enabling each agent to navigate through intersection scenarios without collisions. To achieve this, we designed two intersection scenarios of varying complexities.

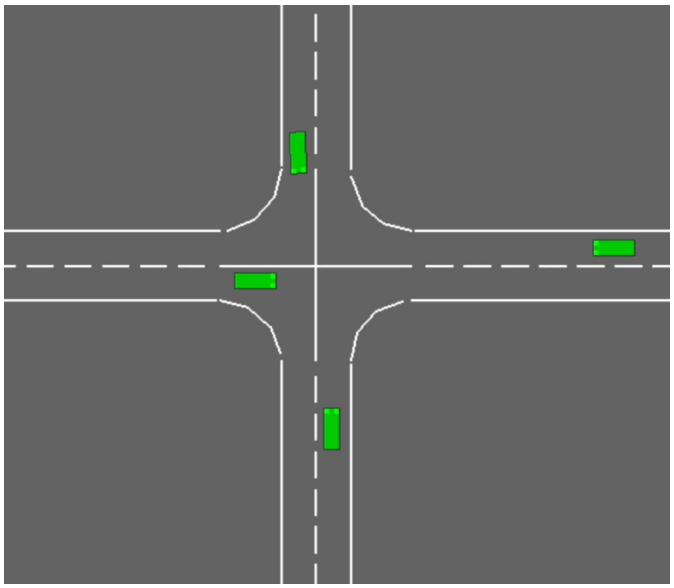

Figure 3.14: Intersection

In the first scenario, as depicted in the figure below, we created a four-way intersection where four autonomous agents approached from different directions. The goal was for each agent to successfully complete their route through the intersection while avoiding any collisions with other vehicles.

In the second scenario, we introduced an additional challenge by adding five random

vehicles representing various human driving patterns to the intersection. This scenario aimed to test the algorithm's ability to handle the presence of unpredictable human-driven vehicles while ensuring the safe navigation of autonomous agents.

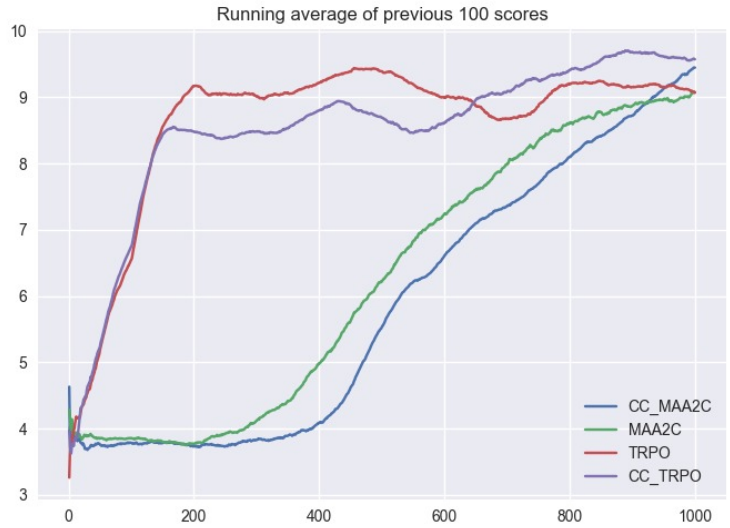

Figure 3.15: Scenario 1: Average Cumulative Return

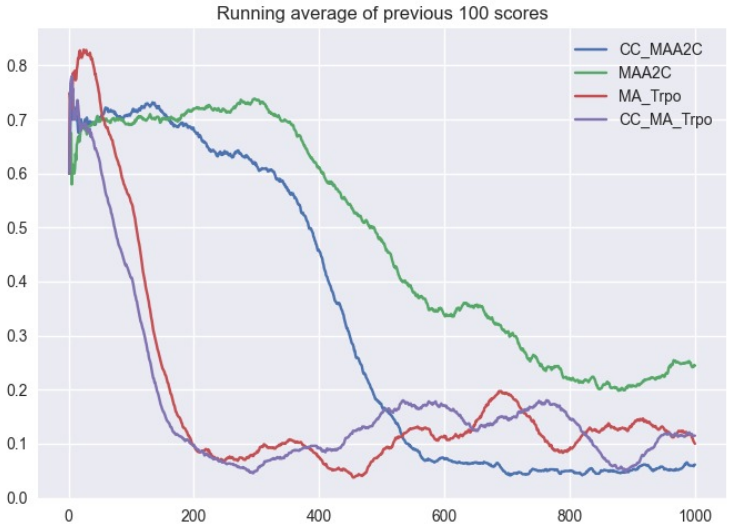

Figure 3.16: Scenario 1: Probability of collision

Algorithms	Cumulative Returns	# Collisions	Probability of Collision
MA-A2C	8.9±0.9	99±10	0.198±0.09
MA-TRPO	9.4±0.9	104±10	0.203±0.2
CC-MAA2C $\Delta = 0.1$	8.1±0.4	24±5	0.05±0.09
CC-MATRPO $\Delta = 0.1$	8.3±0.9	20±3	0.04±0.04

Table 3.5: Intersection Scenario 1 Evaluation Result

Table 3.5 presents the evaluation results for four algorithms, including average cumulative returns, number of collisions, and collision probability. The evaluation was conducted over 10 seeds and 500 iterations, with a 90% confidence interval. MA-A2C achieved an average cumulative reward of approximately 8.9, while MA-TRPO had a slightly higher reward of around 9.4. However, both algorithms had a high number of collisions and, consequently, a high probability of collisions. On the other hand, CC-MAA2C and CC-MATRPO had comparable cumulative rewards to MA-A2C but with a lower number of collisions. Both chance-constrained algorithms successfully met the safety constraint requirements in our evaluations.

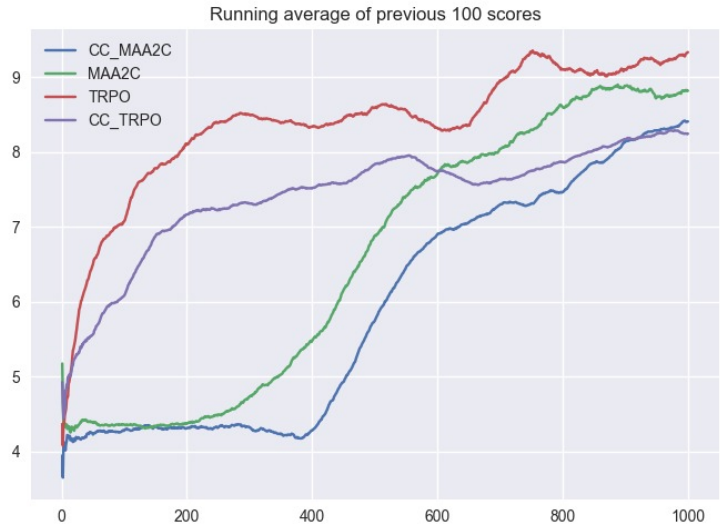

Figure 3.17: Scenario 2:Average Cumulative Return

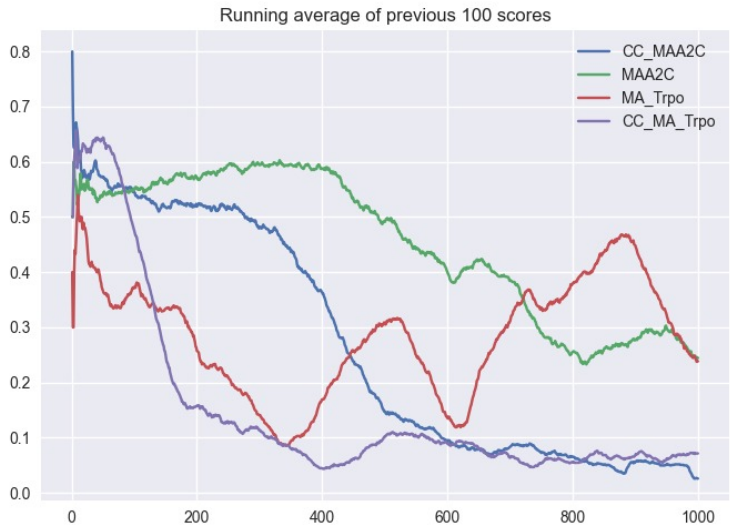

Figure 3.18: Scenario 2:Probability of collision

Table 3.6 presents the evaluation results for four algorithms, including average cumulative returns, number of collisions, and collision probability. The evaluation was conducted over 10 seeds and 500 iterations, with a 90% confidence interval. MA-A2C achieved an average cumulative reward of approximately 8.9, while MA-TRPO had a slightly higher reward of around 9.4. However, both algorithms had a high number of collisions and, consequently, a high probability of collisions. On the other hand, CC-MAA2C and CC-MATRPO had comparable cumulative rewards to MA-A2C but with a lower number of collisions. Both chance-constrained algorithms successfully met the safety constraint requirements in our evaluations.

Algorithms	Cumulative Returns	# Collisions	Probability of Collision
MA-A2C	9.12 ± 0.1	90 ± 5	0.196±0.0
MA-TRPO	9.34 ± 0.0	103 ± 4	0.21 ± 0.11
CC-MAA2C $\Delta = 0.1$	8.8±0.0	20±3	0.04±0.0
CC-MATRPO $\Delta = 0.1$	8.26±0.0	23±4	0.05±0.01

Table 3.6: Intersection Scenario 2 Evaluation Result

CHAPTER 4

Conclusion

4.1 Final Outcomes

In this research paper, we conducted a comprehensive comparative analysis of multiple Multi-Agent Reinforcement Learning (MARL) approaches in highway environments. Our study focused on proposing a single-agent chance constraint method that successfully achieved high cumulative rewards while ensuring relative reasonability. Furthermore, we extended this approach to encompass centralized and decentralized training for multi-agent scenarios. The results demonstrated promising outcomes in terms of satisfying the probability of chance constraints and thresholds, along with achieving satisfactory cumulative rewards. These findings highlight the potential effectiveness of our proposed methodology in addressing the challenges of MARL in highway and Intersection environments

4.2 Limitations

One important limitation of this research is that the proposed approach was only implemented and evaluated using simulations, without conducting tests on real vehicles. While simulations provide a valuable platform for initial exploration and analysis, they may not fully capture the complexities and uncertainties present in real-world driving scenarios. Real-world conditions, such as varying road conditions, unpredictable human drivers, and environmental factors, can significantly impact the performance and effectiveness of the proposed method.

4.3 Future Work

In the future, this work could be extended to incorporate POMDP (Partially Observable Markov Decision Processes). Currently, we only considered MDP (Markov Decision Processes). Additionally, conducting experiments in real-world scenarios would provide valuable insights, as this work primarily focused on simulations. Moreover, an in-depth exploration of reward shaping is crucial for advancing reinforcement learning techniques. Addressing the credit assignment challenge in multi-agent reinforcement learning is also a priority in our future investigations.

Bibliography

[1] Y. Yang and J. Wang, "An overview of multi-agent reinforcement learning from game theoretical perspective," 2021.

[2] E. Leurent, "An environment for autonomous driving decision-making." https://github.com/eleurent/highway-env, 2018.

[3] B. Paden, M. Cap, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," 2016.

[4] C. Katrakazas, M. Quddus, W.-H. Chen, and L. Deka, "Real-time motion planning methods for autonomous on-road driving: state-of-the-art and future research directions," 2 2016.

[5] S. Zhang, W. Deng, Q. Zhao, H. Sun, and B. Litkouhi, "Dynamic trajectory planning for vehicle autonomous driving," in *2013 IEEE International Conference on Systems, Man, and Cybernetics*, pp. 4161–4166, 2013.

[6] L. M. Schmidt, J. Brosig, A. Plinge, B. M. Eskofier, and C. Mutschler, "An introduction to multi-agent reinforcement learning and review of its application to autonomous mobility," 2022.

[7] S. Bhalla, S. G. Subramanian, and M. Crowley, "Deep multi agent reinforcement learning for autonomous driving," in *Canadian Conference on Artificial Intelligence*, vol. LNAI 12109, p. 17, Spring, Lecture Notes in Artificial Intelligence, Spring, Lecture Notes in Artificial Intelligence, 2020.

[8] S. Shalev-Shwartz, S. Shammah, and A. Shashua, "Safe, multi-agent, reinforcement learning for autonomous driving," 2016.

[9] R. Alyassi and M. Khonji, "Dual formulation for chance constrained stochastic shortest path with application to autonomous vehicle behavior planning," in *2021 60th IEEE Conference on Decision and Control (CDC)*, pp. 4486–4492, 2021.

[10] C. Watkins and P. Dayan, "Technical note: Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 05 1992.

[11] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[12] T. Nguyen, N. D. Nguyen, and S. Nahavandi, "Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications," *IEEE Transactions on Cybernetics*, vol. PP, pp. 1–14, 03 2020.

[13] R. S. Sutton, D. Mcallester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems 12*, vol. 12, pp. 1057–1063, MIT Press, 2000.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[15] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proceedings of the 32nd International Conference on Machine*

Learning (F. Bach and D. Blei, eds.), vol. 37 of *Proceedings of Machine Learning Research*, (Lille, France), pp. 1889–1897, PMLR, 07–09 Jul 2015.

[16] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, (Red Hook, NY, USA), p. 6382–6393, Curran Associates Inc., 2017.

[17] L. Weaver and N. Tao, "The optimal reward baseline for gradient-based reinforcement learning," 2013.

[18] T. Zhao, H. Hachiya, G. Niu, and M. Sugiyama, "Analysis and improvement of policy gradient estimation," in *Advances in Neural Information Processing Systems* (J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K. Weinberger, eds.), vol. 24, Curran Associates, Inc., 2011.

[19] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning* (J. Dy and A. Krause, eds.), vol. 80 of *Proceedings of Machine Learning Research*, pp. 1861–1870, PMLR, 10–15 Jul 2018.

[20] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," 2019.

[21] C. Boutilier, "Sequential optimality and coordination in multiagent systems," in *Proceedings of the 16th International Joint Conference on Artifical Intelligence - Volume 1*, IJCAI'99, (San Francisco, CA, USA), p. 478–485, Morgan Kaufmann Publishers Inc., 1999.

[22] R. A. Howard, *Dynamic Programming and Markov Processes*. Cambridge, MA: MIT Press, 1960.

[23] F. Liu and G. Zeng, "A multiagent cooperative learning algorithm," in *Computer Supported Cooperative Work in Design III* (W. Shen, J. Luo, Z. Lin, J.-P. A.

Barthès, and Q. Hao, eds.), (Berlin, Heidelberg), pp. 739–750, Springer Berlin Heidelberg, 2007.

[24] K.-Y. Liang, J. Mårtensson, and K. H. Johansson, "Heavy-duty vehicle platoon formation for fuel efficiency," *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 4, pp. 1051–1061, 2016.

[25] L. Kraemer and B. Banerjee, "Multi-agent reinforcement learning as a rehearsal for decentralized planning," *Neurocomputing*, vol. 190, pp. 82–94, 2016.

[26] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel, "Value-decomposition networks for cooperative multi-agent learning," 2017.

[27] T. Rashid, M. Samvelyan, C. Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning," 03 2018.

[28] T. Rashid, G. Farquhar, B. Peng, and S. Whiteson, "Weighted qmix: Expanding monotonic value function factorisation for deep multi-agent reinforcement learning," 2020.

[29] A. Mahajan, T. Rashid, M. Samvelyan, and S. Whiteson, "Maven: Multi-agent variational exploration," in *Advances in Neural Information Processing Systems* (H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, eds.), vol. 32, Curran Associates, Inc., 2019.

[30] J. N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence*, AAAI'18/IAAI'18/EAAI'18, AAAI Press, 2018.

[31] S. Shalev-Shwartz, S. Shammah, and A. Shashua, "Safe, multi-agent, reinforcement learning for autonomous driving," 2016.

[32] C.-J. Hoel, K. Wolff, and L. Laine, "Automated speed and lane change decision making using deep reinforcement learning," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, pp. 2148–2155, 2018.

[33] C.-J. Hoel, T. Tram, and J. Sjöberg, "Reinforcement learning with uncertainty estimation for tactical decision-making in intersections," 2020.

[34] F. Ye, X. Cheng, P. Wang, C.-Y. Chan, and J. Zhang, "Automated lane change strategy using proximal policy optimization-based deep reinforcement learning," in *2020 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1746–1752, 2020.

[35] B. Mirchevska, C. Pek, M. Werling, M. Althoff, and J. Boedecker, "High-level decision making for safe and reasonable autonomous lane changing using reinforcement learning," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, pp. 2156–2162, 2018.

[36] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," 2013.

[37] K. Jang, E. Vinitsky, B. Chalaki, B. Remer, L. Beaver, A. Malikopoulos, and A. Bayen, "Simulation to scaled city: Zero-shot policy transfer for traffic control via autonomous vehicles," 2019.

[38] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," 2016.

[39] P. Nilsson, O. Hussien, A. Balkan, Y. Chen, A. Ames, J. Grizzle, N. Ozay, H. Peng, and P. Tabuada, "Correct-by-construction adaptive cruise control: Two approaches," *IEEE Transactions on Control Systems Technology*, vol. 24, pp. 1–14, 12 2015.

[40] F. Gritschneder, K. Graichen, and K. Dietmayer, "Fast trajectory planning for automated vehicles using gradient-based nonlinear model predictive control," 2018.

[41] M. Henaff, A. Canziani, and Y. LeCun, "Model-predictive policy learning with uncertainty regularization for driving in dense traffic," 2019.

[42] T. M. Moldovan and P. Abbeel, "Safe exploration in markov decision processes," 2012.

[43] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in ai safety," 2016.

[44] E. Altman, *Constrained Markov Decision Processes*. Chapman and Hall, 1999.

[45] M. Khonji, R. Alyassi, W. Merkt, A. Karapetyan, X. Huang, S. Hong, J. Dias, and B. Williams, "Multi-agent chance-constrained stochastic shortest path with application to risk-aware intelligent intersection," 2022.

[46] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "Carla: An open urban driving simulator," 2017.

[47] D. Krajzewicz, G. Hertkorn, C. Feld, and P. Wagner, "Sumo (simulation of urban mobility); an open-source traffic simulation," pp. 183–187, 01 2002.

[48] M. Zhou, J. Luo, J. Villella, Y. Yang, D. Rusu, J. Miao, W. Zhang, M. Alban, I. Fadakar, Z. Chen, A. C. Huang, Y. Wen, K. Hassanzadeh, D. Graves, D. Chen, Z. Zhu, N. Nguyen, M. Elsayed, K. Shao, S. Ahilan, B. Zhang, J. Wu, Z. Fu, K. Rezaee, P. Yadmellat, M. Rohani, N. P. Nieves, Y. Ni, S. Banijamali, A. C. Rivers, Z. Tian, D. Palenicek, H. bou Ammar, H. Zhang, W. Liu, J. Hao, and J. Wang, "Smarts: Scalable multi-agent reinforcement learning training school for autonomous driving," 2020.

[49] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems* (S. Solla, T. Leen, and K. Müller, eds.), vol. 12, MIT Press, 1999.

[50] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proceedings of the 34th International Conference on Machine Learning* (D. Precup and Y. W. Teh, eds.), vol. 70 of *Proceedings of Machine Learning Research*, pp. 22–31, PMLR, 06–11 Aug 2017.

[51] T.-Y. Yang, J. Rosca, K. Narasimhan, and P. J. Ramadge, "Projection-based constrained policy optimization," 2020.

[52] C. Tessler, D. J. Mankowitz, and S. Mannor, "Reward constrained policy optimization," 2018.

Appendix

REINFORCE Algorithm

The REINFORCE algorithm is outlined below.

Algorithm 7 REINFORCE Algorithm

Input: Policy π_θ with parameters θ
Initialize empty trajectory list: $\mathscr{T}$
Initialize empty gradient list: $\nabla_\theta J$
for episode $\leftarrow$ 1 to N **do**
    Generate episode using policy π_θ: $\tau = (s_1, a_1, r_1, \ldots, s_T, a_T, r_T)$
    Append τ to $\mathscr{T}$
end for
for episode $\leftarrow$ 1 to N **do**
    for $t \leftarrow 1$ to T **do**
        Calculate the action probabilities: $p(a_t|s_t, \theta)$
        Calculate the gradient of the log-probability: $\nabla_\theta \log p(a_t|s_t, \theta)$
        Calculate the return: $G = \sum_{k=t}^{T} \gamma^{k-t} r_k$
        Append $\nabla_\theta \log p(a_t|s_t, \theta) \cdot G$ to $\nabla_\theta J$
    end for
end for
Update the policy parameters: $\theta \leftarrow \theta + \alpha \cdot \nabla_\theta J$